

JS JS > ...

```
1  /**
2   * =====
3   * SCRIPT.JS - Plataforma GOV Digital
4   * Funcionalidades principais e Modo Escuro
5   * =====
6   */
7
8  // =====
9  // VARIÁVEIS GLOBAIS
10 // =====
11
12 let historicoConsultas = [];
13 const STORAGE_KEY_THEME = 'govbr-theme';
14 const STORAGE_KEY_HISTORICO = 'govbr-historico';
15
16 // =====
17 // INICIALIZAÇÃO
18 // =====
19
20 document.addEventListener('DOMContentLoaded', function() {
21     inicializarTema();
22     carregarHistorico();
23     configurarEventos();
24     console.log('Sistema carregado com sucesso!');
25 });
26
27 /**
28 * Inicializa o tema (claro ou escuro) baseado nas preferências do usuário
29 */
30 function inicializarTema() {
31     const temaSalvo = localStorage.getItem(STORAGE_KEY_THEME);
32     const prefereEscuro = window.matchMedia('(prefers-color-scheme: dark)').matches;
```

```
33
34 // Se o usuário tem uma preferência salva, usar ela
35 if (temaSalvo) {
36     if (temaSalvo === 'dark') {
37         ativarModoEscuro();
38     }
39 }
40 // Caso contrário, respeitar a preferência do sistema
41 else if (prefereEscuro) {
42     ativarModoEscuro();
43 }
44
45 // Listener para mudanças de preferência do sistema
46 window.matchMedia('(prefers-color-scheme: dark)').addEventListener('change', (e) => {
47     if (e.matches) {
48         ativarModoEscuro();
49     } else {
50         desativarModoEscuro();
51     }
52 });
53 }
54
55 /**
56  * Configura os event listeners
57  */
58 function configurarEventos() {
59     const themeToggle = document.getElementById('themeToggle');
60     if (themeToggle) {
61         themeToggle.addEventListener('click', alternarTema);
62     }
63
64     // Fechar modal ao clicar fora dele
```

JS JS > alternarTema

```
65     const modal = document.getElementById('modalLogin');
66     if (modal) {
67         modal.addEventListener('click', function(event) {
68             if (event.target === modal) {
69                 fecharModal();
70             }
71         });
72     }
73
74     // Tecla ESC para fechar modal
75     document.addEventListener('keydown', function(event) {
76         if (event.key === 'Escape') {
77             fecharModal();
78         }
79     });
80 }
81
82 // =====
83 // MODO ESCURO
84 // =====
85
86 /**
87  * Alterna entre modo claro e escuro
88  */
89 function alternarTema() {
90     const body = document.body;
91
92     if (body.classList.contains('dark-mode')) {
93         desativarModoEscuro();
94     } else {
95         ativarModoEscuro();
96     }
```



```
97 }
98
99 /**
100  * Ativa o modo escuro
101  */
102 function ativarModoEscuro() {
103     document.body.classList.add('dark-mode');
104     localStorage.setItem(STORAGE_KEY_THEME, 'dark');
105     atualizarIconeTema();
106 }
107
108 /**
109  * Desativa o modo escuro
110  */
111 function desativarModoEscuro() {
112     document.body.classList.remove('dark-mode');
113     localStorage.setItem(STORAGE_KEY_THEME, 'light');
114     atualizarIconeTema();
115 }
116
117 /**
118  * Atualiza o ícone do botão de tema
119  */
120 function atualizarIconeTema() {
121     const themeToggle = document.getElementById('themeToggle');
122     if (!themeToggle) return;
123
124     const isDarkMode = document.body.classList.contains('dark-mode');
125     const icon = themeToggle.querySelector('i');
126
127     if (isDarkMode) {
128         icon.classList.remove('fa-moon');
```

JS JS > ...

```
129     icon.classList.add('fa-sun');
130     themeToggle.title = 'Ativar Modo Claro';
131 } else {
132     icon.classList.remove('fa-sun');
133     icon.classList.add('fa-moon');
134     themeToggle.title = 'Ativar Modo Escuro';
135 }
136 }
137
138 // =====
139 // MODAL
140 // =====
141
142 /**
143  * Abre o modal de login
144  */
145 function abrirModal() {
146     const modal = document.getElementById('modalLogin');
147     if (modal) {
148         modal.classList.add('active');
149         document.body.style.overflow = 'hidden';
150
151         // Focar no primeiro input
152         const cpfInput = document.getElementById('cpfLogin');
153         if (cpfInput) {
154             setTimeout(() => cpfInput.focus(), 100);
155         }
156     }
157 }
158
159 /**
160  * Fecha o modal de login
```

```
161  */
162  function fecharModal() {
163      const modal = document.getElementById('modalLogin');
164      if (modal) {
165          modal.classList.remove('active');
166          document.body.style.overflow = 'auto';
167
168          // Limpar campos
169          document.getElementById('cpfLogin').value = '';
170          document.getElementById('senhaLogin').value = '';
171      }
172  }
173
174  // =====
175  // CONSULTAS E RESULTADOS
176  // =====
177
178  /**
179   * Valida e processa o login
180   */
181  function mostrarResultado() {
182      const cpfInput = document.querySelector('#cpfLogin');
183      const senhaInput = document.querySelector('#senhaLogin');
184
185      if (!cpfInput || !senhaInput) {
186          mostrarAlerta('Erro: Campos não encontrados!', 'erro');
187          return;
188      }
189
190      const cpf = cpfInput.value.trim();
191      const senha = senhaInput.value.trim();
192  }
```



```
JS JS > mostrarSeccaoResultado
193 // Validação básica
194 if (cpf === '' || senha === '') {
195     mostrarAlerta('Preencha CPF e senha corretamente!', 'aviso');
196     return;
197 }
198
199 if (cpf.length < 11) {
200     mostrarAlerta('CPF deve ter pelo menos 11 dígitos!', 'aviso');
201     return;
202 }
203
204 if (senha.length < 6) {
205     mostrarAlerta('Senha deve ter pelo menos 6 caracteres!', 'aviso');
206     return;
207 }
208
209 fecharModal();
210
211 // Adicionar ao histórico
212 adicionarAoHistorico(cpf);
213
214 // Mostrar resultado
215 mostrarSeccaoResultado();
216
217 mostrarAlerta('Consulta realizada com sucesso!', 'sucesso');
218 }
219
220 /**
221  * Mostra a seção de resultado e faz scroll suave
222  * 📌 /
223 function mostrarSeccaoResultado() {
224     const resultado = document.getElementById('resultado');
```

JS JS > ...

```
225     if (resultado) {
226         resultado.classList.add('active');
227         window.scrollTo({
228             top: resultado.offsetTop - 100,
229             behavior: 'smooth'
230         });
231     }
232 }
233
234 /**
235  * Adiciona uma consulta ao histórico
236  */
237 function adicionarAoHistorico(cpf) {
238     const consulta = {
239         cpf: cpf,
240         data: new Date().toLocaleString('pt-BR'),
241         id: Date.now()
242     };
243
244     historicoConsultas.unshift(consulta);
245
246     // Manter apenas os últimos 50 registros
247     if (historicoConsultas.length > 50) {
248         historicoConsultas.pop();
249     }
250
251     salvarHistorico();
252 }
253
254 /**
255  * Carrega o histórico do localStorage
256  */
```



```
257 function carregarHistorico() {
258     const historico = localStorage.getItem(STORAGE_KEY_HISTORICO);
259     if (historico) {
260         try {
261             historicoConsultas = JSON.parse(historico);
262         } catch (e) {
263             console.error('Erro ao carregar histórico:', e);
264             historicoConsultas = [];
265         }
266     }
267 }
268
269 /**
270  * Salva o histórico no localStorage
271  */
272 function salvarHistorico() {
273     localStorage.setItem(STORAGE_KEY_HISTORICO, JSON.stringify(historicoConsultas));
274 }
275
276 // =====
277 // FUNCIONALIDADES INTERATIVAS
278 // =====
279
280 /**
281  * Abre o histórico de consultas
282  */
283 function abrirHistorico() {
284     if (historicoConsultas.length === 0) {
285         mostrarAlerta('Nenhuma consulta foi realizada ainda.', 'info');
286         return;
287     }
288 }
```

```

JS JS > monitorarEmails
289     let historicoTexto = 'HISTÓRICO DE CONSULTAS\n';
290     historicoTexto += '='.repeat(50) + '\n\n';
291
292     historicoConsultas.forEach(function(item, index) {
293         historicoTexto += `Consulta #${index + 1}\n`;
294         historicoTexto += `CPF: ${mascaraCPF(item.cpf)}\n`;
295         historicoTexto += `Data: ${item.data}\n`;
296         historicoTexto += '-'.repeat(50) + '\n\n';
297     });
298
299     // Mostrar em um alert ou modal personalizado
300     alert(historicoTexto);
301 }
302
303 /**
304  * Ativa o monitoramento de e-mails
305  */
306 function monitorarEmails() {
307     const email = prompt('Digite seu e-mail para ativar o monitoramento:');
308
309     if (email === null) {
310         return;
311     }
312
313     if (email === '') {
314         mostrarAlerta('E-mail não pode estar vazio!', 'aviso');
315         return;
316     }
317
318     if (!validarEmail(email)) {
319         mostrarAlerta('E-mail inválido!', 'aviso');
320         return;

```

```
321     }
322
323     mostrarAlerta(`Monitoramento ativado para: ${email}`, 'sucesso');
324 }
325
326 /**
327  * Ativa o modo mobile
328  */
329 function abrirMobile() {
330     const isMobileMode = document.body.style.maxWidth === '480px';
331
332     if (isMobileMode) {
333         document.body.style.maxWidth = 'none';
334         document.body.style.margin = '0';
335         mostrarAlerta('Modo mobile desativado!', 'info');
336     } else {
337         document.body.style.maxWidth = '480px';
338         document.body.style.margin = '0 auto';
339         mostrarAlerta('Modo mobile ativado!', 'info');
340     }
341 }
342
343 /**
344  * Ativa o login biométrico
345  */
346 function loginBiometrico() {
347     const autenticado = confirm('Deseja autenticar utilizando biometria?');
348
349     if (autenticado) {
350         // Simular validação biométrica
351         mostrarAlerta('Biometria validada com sucesso!', 'sucesso');
352     } else {
```


JS .JS > antiFraude > setTimeout() callback

```
353     mostrarAlerta('Autenticação cancelada.', 'info');
354 }
355 }
356
357 /**
358  * Inicia o backup seguro
359  */
360 function backupSeguro() {
361     mostrarAlerta('Iniciando backup criptografado...', 'info');
362
363     setTimeout(function() {
364         mostrarAlerta('Backup concluído com sucesso!', 'sucesso');
365     }, 2000);
366 }
367
368 /**
369  * Ativa a proteção anti-fraude
370  */
371 function antiFraude() {
372     const itens = document.querySelectorAll('.result-item');
373
374     itens.forEach(function(item) {
375         item.style.background = '#d8ffe1';
376         item.style.borderLeft = '4px solid #0d7c2f';
377     });
378
379     mostrarAlerta('Proteção Anti Fraude ativada!', 'sucesso');
380
381     // Remover destaque após 5 segundos
382     setTimeout(function() {
383         itens.forEach(function(item) {
384             item.style.background = '';
```

```

JS JS > ...
385         item.style.borderLeft = '';
386     });
387 }, 5000);
388 }
389
390 /**
391  * Gera um relatório de segurança
392  */
393 function gerarRelatorio() {
394     const data = new Date().toLocaleString('pt-BR');
395     const texto = `RELATÓRIO DE SEGURANÇA\n`;
396     texto += `Data: ${data}\n`;
397     texto += `Status: Sistema ativo e protegido\n`;
398     texto += `Consultas realizadas: ${historicoConsultas.length}\n`;
399     texto += `Modo: ${document.body.classList.contains('dark-mode')} ? 'Escuro' : 'Claro'\n`;
400
401     const blob = new Blob([texto], { type: 'text/plain;charset=utf-8' });
402     const link = document.createElement('a');
403
404     link.href = URL.createObjectURL(blob);
405     link.download = `relatorio-seguranca-${Date.now()}.txt`;
406
407     document.body.appendChild(link);
408     link.click();
409     document.body.removeChild(link);
410
411     mostrarAlerta('Relatório gerado com sucesso!', 'sucesso');
412 }
413
414 /**
415  * Atualiza o painel de estatísticas
416  */

```

```
417 function atualizarPainel() {
418     const stats = document.querySelectorAll('.stat h2');
419
420     if (stats.length >= 3) {
421         stats[0].innerHTML = (Math.random() * 5).toFixed(1) + 'M';
422         stats[1].innerHTML = (90 + Math.random() * 10).toFixed(1) + '%';
423         stats[2].innerHTML = '24/7';
424     }
425
426     mostrarAlerta('Painel atualizado com sucesso!', 'sucesso');
427 }
428
429 // =====
430 // FUNÇÕES UTILITÁRIAS
431 // =====
432
433 /**
434  * Valida um e-mail
435  */
436 function validarEmail(email) {
437     const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
438     return regex.test(email);
439 }
440
441 /**
442  * Mascara um CPF para exibição
443  */
444 function mascaraCPF(cpf) {
445     if (!cpf || cpf.length < 11) return cpf;
446     return cpf.substring(0, 3) + '.*.*.*' + cpf.substring(cpf.length - 2);
447 }
448
```


JS JS > ...

```
449 /**
450  * Mostra um alerta personalizado
451  */
452 function mostrarAlerta(mensagem, tipo = 'info') {
453     // Tipos: 'sucesso', 'erro', 'aviso', 'info'
454     const cores = {
455         'sucesso': '#0d7c2f',
456         'erro': '#c41e3a',
457         'aviso': '#ff9800',
458         'info': '#1351b4'
459     };
460
461     // Por enquanto, usar alert padrão
462     // Em uma aplicação real, você criaria um toast notification
463     alert(`[${tipo.toUpperCase()}] ${mensagem}`);
464 }
465
466 /**
467  * Copia texto para a área de transferência
468  */
469 function copiarParaClipboard(texto) {
470     navigator.clipboard.writeText(texto).then(() => {
471         mostrarAlerta('Copiado para a área de transferência!', 'sucesso');
472     }).catch(() => {
473         mostrarAlerta('Erro ao copiar para a área de transferência!', 'erro');
474     });
475 }
476
477 /**
478  * Formata um número como moeda
479  */
480 function formatarMoeda(valor) {
```

JS .JS >  debounce >  <function> >  setTimeout() callback

```
481     return new Intl.NumberFormat('pt-BR', {
482         style: 'currency',
483         currency: 'BRL'
484     }).format(valor);
485 }
486
487 /**
488  * Formata uma data
489  */
490 function formatarData(data) {
491     return new Intl.DateTimeFormat('pt-BR', {
492         year: 'numeric',
493         month: 'long',
494         day: 'numeric',
495         hour: '2-digit',
496         minute: '2-digit'
497     }).format(new Date(data));
498 }
499
500 // =====
501 // PERFORMANCE E OTIMIZAÇÕES
502 // =====
503
504 /**
505  * Debounce para otimizar eventos
506  */
507 function debounce(func, delay) {
508     let timeoutId;
509     return function(...args) {
510         clearTimeout(timeoutId);
511          timeoutId = setTimeout(() => func.apply(this, args), delay);
512     };
513 }
```

```

513 }
514
515 /**
516  * Throttle para limitar execução de funções
517  */
518 function throttle(func, limit) {
519     let inThrottle;
520     return function(...args) {
521         if (!inThrottle) {
522             func.apply(this, args);
523             inThrottle = true;
524             setTimeout(() => inThrottle = false, limit);
525         }
526     };
527 }
528
529 // =====
530 // LOGS E DEBUG
531 // =====
532
533 console.log('%cPlataforma GOV Digital', 'font-size: 20px; font-weight: bold; color: #1351b4;');
534 console.log('%cVersão 1.0.0', 'color: #666;');
535 console.log('%cModo Escuro: Ativado', 'color: #4caf50;');P

```